Widgets and layout

Composition, constraints, scrolling, and Material scaffolding

Dinu-Ștefan Rusu ADMD

What you should leave with

Widget trees

Explain what a widget is, why it is immutable, and which of the three trees holds state.

Layout

Use Container, Padding, Row, Column, Expanded and Stack, and read the constraint rules.

Scrolling lists

Choose between ListView, ListView.builder, GridView and SingleChildScrollView.

Material screens

Wrap content in a Scaffold with an AppBar, a floating action button, and a SnackBar.

1

Widgets

Immutability, the three trees, composition, rebuilds

Contents Page

1 **Widgets** 3
2 Layout 12
3 Scrolling 20
4 Material components 24
5 Lab 3 todo screen 29

Widgets

A description of UI

A widget is an immutable description of a piece of screen. It is not the pixels.

Nesting is the tree

A Scaffold holds a Column, which holds a Text and a button. That nesting is the widget tree.

Cheap and disposable

Widgets are created and thrown away constantly. Creating one allocates a small object, nothing more.

Structure is a widget too

Padding, Center, Row and Column are widgets. So is the whole app.

Style is a widget too

Opacity, DecoratedBox and Transform are widgets. There is no separate style language.

Behavior is a widget too

GestureDetector and Focus are widgets. Interaction is wrapped around content.

Widget immutability

```
// Every field is final. Nothing in a
// widget is edited after construction.
const Text('Todos')   // canonicalized:
                      // same instance
                      // every build

Text('Todos')         // a new instance
                      // every build
```

Change means rebuild.
Changing the screen means constructing a new description, never editing the old one.
Comparing two immutable descriptions is fast, so the framework can skip whole subtrees that did not change. Write `const` wherever the analyzer allows it.

Immutability is what makes rebuilding cheap. A rebuild hands Flutter a new blueprint. It does not repaint the screen.

The three trees

Every Flutter app keeps three parallel trees. Almost every rebuild question is answered by naming which one you mean.

Widget

Immutable configuration. Created and thrown away on every build. You write this one.

Element

One per position in the tree. Holds your State object. Decides whether to reuse or rebuild.

RenderObject

Layout, paint and hit-testing. Expensive to create, so it is mutated in place.

You build the left tree. The middle tree decides how little of the right tree has to change. The cost model is MEC Lecture 13.

Composition over inheritance

```
// You do not subclass Text to give it
// padding and a border. You wrap it.
Padding(
  padding: const EdgeInsets.all(16),
  child: DecoratedBox(
    decoration: BoxDecoration(
      border: Border.all(
        color: Colors.black12,
      ),
    ),
    child: const Text('Hello'),
  ),
)
```

One widget, one job.
Layout comes from nesting small widgets.
That is why trees get deep. Deep trees are normal and cheap.

StatelessWidget

```
class Greeting extends StatelessWidget {
  const Greeting({
    super.key,
    required this.name,
  });

  final String name;  // always final

  @override
  Widget build(BuildContext context) {
    return Text('Hello, $name');
  }
}
```

It does not build once.
A StatelessWidget rebuilds whenever its parent rebuilds, or whenever something it reads changes.
What it does not have is mutable state of its own. Everything it needs arrives through its constructor. Give it a `const` constructor whenever you can.

StatefulWidget and setState

```
class CounterView extends StatefulWidget {
  const CounterView({super.key});
  @override
  State<CounterView> createState() => _CounterViewState();
}
class _CounterViewState extends State<CounterView> {
  int _count = 0;                    // the mutable state
  @override
  Widget build(BuildContext context) => TextButton(
      onPressed: () => setState(() => _count++),
      child: Text('Count: $_count'));
}
```

Two classes, one widget. The State object survives every parent rebuild.

Rebuild scope and keys

Flutter does not rebuild the whole tree each frame. `setState` marks one element dirty, and only that subtree rebuilds.

On rebuild, new widgets are matched against existing elements by runtime type first, then by key. A match reuses the element and its State. No match discards both.

Keys

`ValueKey(id)` takes identity from a value.

`UniqueKey()` never matches, so it forces a fresh element.

You need a key when list items are reordered, inserted or removed. Otherwise State follows position, not item.

Introduced here, finished in Lecture 5. Today: keys exist and lists need them. Lecture 5 covers the full reconciliation rules.

Extracting widgets

```
class UserListItem extends StatelessWidget {
  const UserListItem(this.user,
      {super.key});
  final User user;
  @override
  Widget build(BuildContext context) => Row(
        children: [
          Avatar(url: user.avatarUrl),
          const SizedBox(width: 12),
          Expanded(child: UserInfo(user)),
        ],
      );
}
```

Two rules of thumb.
Extract when a subtree repeats, and extract when `build` stops fitting on your screen.
Small widgets rebuild in smaller pieces, so extraction is a performance tool too. Prefer a named widget class over a method that returns a Widget. Only the class gets its own element.

2

Layout

Constraints, Container, Row, Column, Expanded, Stack, overflow errors

Contents

		Page
1	Widgets	3
2	**Layout**	12
3	Scrolling	20
4	Material components	24
5	Lab 3 todo screen	29

Layout constraints

↓ 1. Constraints go down

A parent tells each child the minimum and maximum width and height it may take.

↑ 2. Sizes go up

The child picks its own size inside those limits and reports it back to the parent.

⌖ 3. Parent sets position

The child never chooses where it sits. The parent places it in its own coordinate space.

One pass down, one pass up. Layout is a single walk of the tree, which is why it is fast and why it cannot backtrack.

Padding, SizedBox and Container

```
// Reach for these two first.
const Padding(
  padding: EdgeInsets.all(16),
  child: Text('Todos'),
)
const SizedBox(height: 12)   // a gap
// Container bundles several widgets.
Container(
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(color: card),
  child: const Text('Todos'),
)
```

Container is a convenience.
It wraps Padding, DecoratedBox, ConstrainedBox and Align in one constructor.
Use it when you need most of them. Use Padding or SizedBox when you need one.

Gaps

A `SizedBox` with only a height is the idiomatic spacer.

Row and Column

```
Column(
  mainAxisAlignment:
      MainAxisAlignment.center,
  crossAxisAlignment:
      CrossAxisAlignment.start,
  mainAxisSize: MainAxisSize.min,
  children: const [
    Text('Todos'),
    SizedBox(height: 8),
    Text('3 items left'),
  ],
)
```

Same widget, two axes.
Row lays children out horizontally, Column vertically. Everything else is identical.
`mainAxisAlignment` moves children along the axis. `crossAxisAlignment` moves them across it.
`MainAxisSize.min` shrinks the Row or Column to its children instead of taking all the space offered.

Expanded and Flexible

```
Row(children: [
  const Icon(Icons.person),
  const SizedBox(width: 8),
  // Takes all the leftover width.
  Expanded(child: Text(user.name)),
  // Takes twice as much as flex: 1.
  Expanded(flex: 2, child: Chart()),
])
```

They divide leftover space.
The Row first sizes its unflexible children, then splits what is left by flex factor.
`Expanded` forces the child to fill its share. `Flexible` allows a smaller child. Both work only as a direct child of a Row, Column or Flex. Anywhere else they throw.

Stack, Positioned and Align

```
Stack(
  children: [
    const Avatar(),    // painted first
    const Positioned(
      right: 0, bottom: 0,
      child: OnlineDot(),
    ),
    const Align(
      alignment: Alignment.topCenter,
      child: Text('online'),
    ),
  ],
)
```

Overlay, not flow.
A Stack paints its children on top of each other. The first child is at the bottom.
`Positioned` pins a child to one or more edges. Only a direct child may be Positioned. `Align` places a child by a fraction of the parent box, which reads better than four offsets.

Common layout widgets

Widget	What it does	The knobs you turn
Container	padding, decoration, size	padding, decoration, alignment
Row / Column	children along one axis	main and cross axis alignment
Expanded / Flexible	split leftover space	flex: fill versus may shrink
Stack / Positioned	overlay children	top, left, right, bottom
SizedBox	impose a size or a gap	width, height, constraints
ListView / GridView	scroll a list or a grid	builder, separated, direction

Keep this table open during the lab.

RenderFlex overflow errors

```
// Breaks: the Text asks for more width
// than the Row can give it.
Row(children: [
  const Icon(Icons.person),
  Text(user.veryLongDisplayName),
])

// Fixes: give the Text a share
Row(children: [
  const Icon(Icons.person),
  Expanded(child: Text(user.name)),
])
```

Read the box

The yellow and black stripes name the widget and the axis. The console names the file and line.

Three usual fixes: wrap the child in `Expanded`, wrap it in `Flexible`, or make the Row scroll. For text, `maxLines` with an ellipsis overflow is often what you want.

3

Scrolling

ListView, builders, grids, scroll views

Contents Page

1 Widgets 3
2 Layout 12
3 Scrolling 20
4 Material components 24
5 Lab 3 todo screen 29

ListView and ListView.builder

```
ListView(   // eager: every child built now
  children: [
    for (final t in todos)
      TodoTile(todo: t),
  ],
)

ListView.builder(   // lazy: visible slice
  itemCount: todos.length,
  itemBuilder: (context, i) => TodoTile(
    key: ValueKey(todos[i].id),
    todo: todos[i],
  ),
)
```

The builder is lazy.
It calls `itemBuilder` only for items near the viewport, and discards them as they leave.
`ListView.separated` is the same thing with a divider between items. A list of ten fixed rows can use the eager form. Anything that grows with your data must be a builder.

GridView and SingleChildScrollView

```
// Lazy in two dimensions: only visible cells are ever built.
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    childAspectRatio: 1.4,
  ),
  itemCount: photos.length,
  itemBuilder: (context, i) => PhotoTile(photos[i]),
)
// Scrolls exactly one child, and builds all of it.
SingleChildScrollView(child: Column(children: fields))
```

One scrollable per axis per screen. Nesting two vertical scroll views is a bug.

Choosing a scrolling widget

You have	Use	Why
A long form	SingleChildScrollView	every field must exist
A feed or a data list	ListView.builder	only visible rows are built
A list with dividers	ListView.separated	a builder runs between items
A photo or card grid	GridView.builder	lazy, with a delegate per cell
A collapsing header	CustomScrollView	header and list share one scroll
A fixed short list	Column in a scroll view	no laziness needed at ten rows

Slivers are the low-level pieces every scroll view is built from. You need them only for the fifth row.

4

Material components

Scaffold, AppBar, buttons, SnackBars and dialogs

Contents

		Page
1	Widgets	3
2	Layout	12
3	Scrolling	20
4	**Material components**	24
5	Lab 3 todo screen	29

Scaffold

```
Scaffold(
  appBar: AppBar(
    title: const Text('Todos'),
    actions: const [SearchButton()],
  ),
  body: const TodoList(),
  floatingActionButton:
      FloatingActionButton(
        onPressed: _addTodo,
        child: const Icon(Icons.add),
      ),
  bottomNavigationBar: const HomeNavBar(),
)
```

One Scaffold per screen.
It gives you the app bar, the body, the floating button, the drawer and the SnackBar host.
Every slot is optional. A screen with only a body is a valid Scaffold. The Scaffold also keeps your content clear of the keyboard and the system bars.

AppBar, buttons and navigation bars

AppBar

Title, actions, and a back button that appears on its own when the route can pop.

FloatingActionButton

One primary action per screen. Add a todo, compose a message, start a scan.

Buttons

FilledButton for the main action, OutlinedButton and TextButton for the rest, IconButton in bars.

NavigationBar

The Material 3 bottom bar. Three to five destinations, one selected index.

Drawer

A side panel for secondary destinations. Pass it to the Scaffold and the AppBar adds the handle.

ThemeData

Colors and typography come from the theme, not from each widget. Lecture 14 goes deeper.

SnackBars and dialogs

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text('Todo deleted'),
    action: SnackBarAction(
        label: 'Undo', onPressed: _restore),
  ),
);
final ok = await showDialog<bool>(
  context: context,
  builder: (_) => const ConfirmDelete(),
);
```

Two different jobs.
A SnackBar reports what already happened. A dialog blocks until the user decides.
`showDialog` returns a Future. It completes with whatever `Navigator.pop` was given.

Async gap

After an `await`, check `context.mounted` before using the context again.

Cupertino and adaptive widgets

Cupertino exists

CupertinoApp, CupertinoPageScaffold, CupertinoNavigationBar and CupertinoButton mirror the iOS look.

Adaptive constructors

Switch.adaptive and CircularProgressIndicator.adaptive pick the platform look at runtime.

Pick one and commit

Most teams ship Material on both platforms.

This course uses Material. Page transitions already adapt. The labs are graded on Material widgets.

5

Lab 3 todo screen

Screen state, list, TodoTile, adding and removing items, empty state

Contents Page

1 Widgets 3
2 Layout 12
3 Scrolling 20
4 Material components 24
5 Lab 3 todo screen 29

Step 1: the screen and its state

```
class Todo {
  Todo(this.id, this.title);
  final int id;
  final String title;
  bool done = false;
}
class TodoPage extends StatefulWidget {
  const TodoPage({super.key});
  @override
  State<TodoPage> createState() =>
      _TodoPageState();
}
```

Start from the data.
One plain Dart class holds a todo. There are no framework types in it. The screen is stateful because the list of todos changes while the screen is on the tree. In Lecture 6 this same list moves into a Cubit and the screen goes back to stateless.

Step 2: the list

```
class _TodoPageState extends State<TodoPage> {
  final _todos = <Todo>[];
  @override
  Widget build(BuildContext context) => Scaffold(
        body: ListView.builder(
          itemCount: _todos.length,
          itemBuilder: (context, i) => TodoTile(
              key: ValueKey(_todos[i].id), todo: _todos[i],
              onToggle: () => _toggle(_todos[i])),
        ),
      );
}
```

The `ValueKey` ties State to the todo, not to the row position.

Step 3: extract TodoTile

```
class TodoTile extends StatelessWidget {
  const TodoTile({super.key, required this.todo, required this.onToggle});
  final Todo todo;
  final VoidCallback onToggle;

  @override
  Widget build(BuildContext context) => ListTile(
        leading: Checkbox(
            value: todo.done, onChanged: (_) => onToggle()),
        title: Text(todo.title),
      );
}
```

Data flows down the constructor, events flow up the callback.

Step 4: adding and removing an item

```dart
int _nextId = 0;

void _add(String title) {
  final todo = Todo(_nextId++, title);
  setState(() => _todos.add(todo));
}

void _toggle(Todo todo) => setState(() => todo.done = !todo.done);

void _remove(Todo todo) => setState(() => _todos.remove(todo));
```

Mutate, then setState. Every change to the list happens inside the callback, so Flutter schedules a frame. Change the list outside it and the screen never updates.

Step 5: the empty state and a SnackBar

```
body: _todos.isEmpty
    ? const Center(
        child: Text('Nothing to do yet.'),
      )
    : ListView.builder(...),

// after _remove(todo)
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('Removed')),
);
```

Empty, loading and error states are part of the design.
Today the empty state is a Text. In Lecture 6 the same branch becomes a switch over bloc states.

Lab 3

You build this screen, then add swipe to delete.

Summary

1. **Widgets are immutable descriptions.** You compose small ones instead of configuring big ones.
2. **Constraints go down, sizes go up, the parent sets position.** Every layout surprise comes from one of those three.
3. **If the list grows with your data, it has to be a builder.** Eager lists build everything before the first frame.

Further reading

Flutter widget catalog · Understanding constraints · Scrolling

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel